ITI · AI Agents Development using Python track

Python Programming for AI Applications

Eng. Fatma Elraey

Lab 4 — Classes, Inheritance & Custom Data Structures

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications (6-Day Course)
Session	Lab 4 of 6
Focus	Building a Queue class from scratch, extending it through inheritance, tracking instances, and saving/loading state — the same object patterns behind every data pipeline buffer.

Before You Start

This lab builds directly on today's lecture: you'll define a class with a constructor, add instance methods that operate on self, extend that class through inheritance, use a class variable to track every instance ever created, and raise your own custom exception when something goes wrong — the same object-oriented patterns behind a real-time data buffer or a job queue in an ML pipeline.

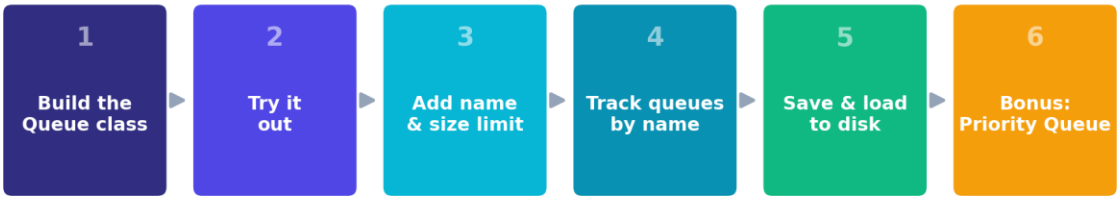

Figure L.1 — Six steps: build the queue, try it out, add limits, track instances, persist to disk, then the bonus round.

✓ QUICK CHECK — Before You Start

Q1. What is the first parameter of every instance method, by convention?

Answer: *self — it refers to the specific object the method is being called on.*

Q2. How does a class variable differ from an instance variable?

Answer: *A class variable is defined in the class body and shared across every instance; an instance variable is set with self.x and belongs to just one object.*

Q3. What keyword lets a custom exception carry your own message?

Answer: *raise — e.g. raise ValueError("your message here"), or raise MyCustomError("message") for your own exception class.*

01 Build a Queue Class

A queue is a linear data structure that follows First In, First Out (FIFO) order — the first value inserted is the first one removed, exactly like a line of people waiting for a resource. Implement a Queue class with three operations: insert(value), pop(), and is_empty().

- *insert(value) adds a new value at the rear of the queue.*
- *pop() removes and returns the value at the front of the queue. If the queue is empty, print a warning message and return None instead of raising an error.*
- *is_empty() returns True if the queue has no items, and False otherwise.*
- *Store the items in a plain list inside __init__, and use list operations (append, pop(0)) to implement insert and pop.*

AI connection: *Every streaming data pipeline needs somewhere to hold items that have arrived but not yet been processed — a queue like this one is the simplest version of that buffer.*

02 Try Your Queue

Write a short program that creates a Queue, inserts a few sensor readings into it, pops them back out in order, and pops one extra time to confirm the empty-queue warning works.

- *Reuse the Queue class from Exercise 01 (put it in the same file, or import it if you saved it separately).*
- *Insert at least three values, then pop them one at a time and print each result.*
- *Call pop() one more time after the queue is empty and confirm you see the warning instead of a crash.*

AI connection: *This is exactly how you'd sanity-check any buffer before trusting it inside a bigger pipeline — confirm the happy path and the edge case both behave as expected.*

Expected output:

```
101
102
103
Warning: queue is empty
```

03 Add a Name and a Size Limit

Create a new class NamedQueue that behaves like Queue but adds two things through its constructor: a name, and a maximum size. If someone tries to insert past that size, raise a custom exception instead of silently growing forever.

- *NamedQueue's __init__ should take (self, name, size) and set up the same empty list Queue used.*
- *Define your own exception class QueueOutOfRangeException(Exception) above NamedQueue.*
- *Override insert(value) so it raises QueueOutOfRangeException("queue '<name>' is full") when the queue is already at size, otherwise behaves like the normal insert.*
- *pop() and is_empty() can be reused as-is — you don't need to redefine them.*

AI connection: *A bounded queue like this is exactly how a producer/consumer buffer protects a model-serving pipeline from being flooded faster than it can process requests.*

04 Track Every Queue by Name

Extend NamedQueue so the class itself keeps a record of every queue instance ever created, keyed by name — and add a class method that can fetch any of them back by that name.

- *Add a class variable, e.g. registry = {}, defined once in the class body (not inside __init__).*
- *Inside __init__, after setting self.name, store the new instance in the registry: NamedQueue.registry[name] = self.*
- *Add a classmethod get_queue(cls, name) that returns cls.registry.get(name).*
- *Create two or three named queues, then fetch one back by name and confirm it's the same object (use is or compare its .name).*

AI connection: *This mirrors how a scheduler keeps track of every active job queue by name, so any part of the system can look one up without passing the object around everywhere.*